

Informe Técnico

Sistema de Gestión de Eventos

Análisis de funcionamiento y problemas encontrados durante el
desarrollo

Tecnologías: Python, Django, Django
REST Framework, Bootstrap

Patrones utilizados: Singleton, Builder,
Prototype, Adapter, Composite, Chain of
Responsibility, Bridge

11 de abril de 2026

Índice

1. Introducción	2
1.1. Objetivo del sistema	2
2. Arquitectura general del proyecto	2
3. Funcionamiento funcional del sistema (explicación sencilla)	3
3.1. Panel principal	3
3.2. Creación de eventos	3
3.3. Clonado de eventos	4
3.4. Configuración global	4
3.5. API REST	4
4. Patrones de diseño aplicados y su valor	4
4.1. Singleton (singleton.py)	4
4.2. Builder (builder.py)	4
4.3. Prototype (prototype.py)	4
4.4. Adapter (practica2.py)	5
4.5. Composite (practica2.py)	5
4.6. Chain of Responsibility (practica2.py)	5
4.7. Bridge (practica2.py)	5
5. Problemas detectables durante el desarrollo	5
5.1. Inconsistencia entre tests y código real	5
5.2. Persistencia en memoria no apta para producción	5
5.3. Riesgos de concurrencia	6
5.4. Validaciones incompletas	6
5.5. Tipado y compatibilidad de versiones	6
5.6. Mantenibilidad del front	6
6. Flujo de ejecución resumido	6
6.1. Crear conferencia	6
6.2. Exportar JSON	7
7. Conclusión	7

1. Introducción

Aquí se explica el funcionamiento de cada funcionalidad del sistema de gestión de eventos implementado en Django. Además, se analizan los principales problemas que aparecen al programar este tipo de solución, cuando se combinan múltiples patrones de diseño.

El proyecto presenta un enfoque didáctico: no usa base de datos persistente tradicional para los eventos, sino una estructura en memoria (`dict`), lo cual simplifica la demostración de patrones y acelera pruebas en entorno local.

1.1. Objetivo del sistema

El objetivo principal es permitir:

- Crear eventos pre-configurados según tipo (conferencia, boda, concierto, cena de empresa).
- Clonar eventos existentes.
- Validar eventos antes de guardarlos.
- Exportar la información en HTML o JSON.
- Gestionar configuración global compartida (moneda, impuestos) con una sola instancia.

2. Arquitectura general del proyecto

El proyecto está dividido en módulos con responsabilidades bien separadas:

Archivo/Módulo	Responsabilidad principal
<code>entidades.py</code>	Define modelos de dominio con <code>dataclass</code> : <code>Evento</code> , <code>Servicio</code> , <code>Ubicacion</code> , <code>Usuario</code> .
<code>memoria.py</code>	Simula una “base de datos” en memoria con diccionarios y secuencias autoincrementales.
<code>builder.py</code>	Construcción de eventos por tipo usando Builder + Director.
<code>prototype.py</code>	Clonación profunda de eventos (Prototype).
<code>singleton.py</code>	Configuración global única compartida (Singleton).
<code>practica2.py</code>	Implementa Adapter, Composite, Chain of Responsibility y Bridge.

<code>views.py</code>	Controla flujo web: creación, clonación, panel principal y configuración.
<code>api.py</code>	Endpoints REST para listar y consultar eventos.
<code>dashboard.html</code>	Interfaz visual Bootstrap para mostrar y accionar funcionalidades.
<code>tests.py</code>	Pruebas unitarias simples para validar comportamiento esperado.

3. Funcionamiento funcional del sistema (explicación sencilla)

3.1. Panel principal

La vista `panel_principal` hace lo siguiente:

1. Obtiene la configuración global (moneda e impuestos) desde Singleton.
2. Toma todos los eventos guardados en memoria.
3. Evalúa si se pidió exportación JSON con `?export=json`.
4. Si se pidió JSON, usa Bridge para devolver datos serializables.
5. Si no, usa Bridge para preparar contexto HTML y renderizar `dashboard.html`.

3.2. Creación de eventos

Desde la interfaz, cada tarjeta dispara una URL del tipo:

`/crear/conferencia/`, `/crear/boda/`, etc.

La vista `crear_evento_builder`:

- Selecciona el Builder concreto según tipo.
- Usa `DirectorEvento` para ejecutar una receta de construcción.
- Obtiene el objeto final.
- Lo pasa por una cadena de validación.
- Si es válido, lo guarda y redirige al panel.

3.3. Clonado de eventos

La ruta de clonado:

1. Busca evento por id.
2. Si no existe, devuelve 404.
3. Si existe, usa `clonar()` (deep copy).
4. Limpia el id para que al guardar reciba nuevo identificador.
5. Cambia nombre a “Copia de ...”.

3.4. Configuración global

La ruta `/configuracion/` alterna moneda e impuestos:

- EUR/21 % ↔ USD/10 %.

Como es Singleton, todos los componentes ven el cambio inmediatamente.

3.5. API REST

`EventoViewSet` expone:

- GET `/api/v1/eventos/`: lista todos.
- GET `/api/v1/eventos/{id}/`: detalle por id.

4. Patrones de diseño aplicados y su valor

4.1. Singleton (`singleton.py`)

Qué hace: asegura una única instancia de configuración global. **Ventaja:** evita inconsistencias de valores globales en distintas partes del sistema. **Riesgo:** en pruebas o concurrencia puede introducir estado compartido difícil de resetear.

4.2. Builder (`builder.py`)

Qué hace: encapsula construcción compleja de eventos por tipo. **Ventaja:** separar *cómo se construye* de *qué se usa*. **Riesgo:** si cambian nombres de servicios y tests no se actualizan, fallan pruebas aunque la lógica sea correcta.

4.3. Prototype (`prototype.py`)

Qué hace: duplica eventos con `deepcopy`. **Ventaja:** reutiliza estructuras completas rápidamente. **Riesgo:** copiar objetos muy grandes puede ser costoso en memoria/tiempo.

4.4. Adapter (`practica2.py`)

Qué hace: adapta datos de API externa de streaming al modelo interno **Servicio**. **Ventaja:** desacopla proveedor externo de dominio interno. **Riesgo:** si cambia el contrato del proveedor (campos JSON), falla adaptación.

4.5. Composite (`practica2.py`)

Qué hace: agrupa múltiples servicios como un único servicio “pack”. **Ventaja:** tratamiento homogéneo de elementos simples y compuestos. **Riesgo:** perder trazabilidad de ítems internos si solo se persiste resumen agregado.

4.6. Chain of Responsibility (`practica2.py`)

Qué hace: valida en cadena (fecha, capacidad, etc.). **Ventaja:** validaciones modulares y reordenables. **Riesgo:** orden incorrecto de manejadores puede ocultar errores.

4.7. Bridge (`practica2.py`)

Qué hace: separa abstracción de representación (HTML vs JSON). **Ventaja:** agregar formatos nuevos sin tocar la lógica principal. **Riesgo:** sobreingeniería si solo habrá un formato.

5. Problemas detectables durante el desarrollo

5.1. Inconsistencia entre tests y código real

En `tests.py` se espera que exista el servicio "Streaming 4K", pero el adapter crea:

"Streaming Externo (4K)"

Esto produce fallo de test por nombre distinto.

Solución sugerida:

- O bien ajustar test para buscar el nombre real.
- O bien adaptar el adapter para devolver exactamente el nombre esperado por test.

5.2. Persistencia en memoria no apta para producción

`db_eventos` vive en RAM:

- Se pierde todo al reiniciar servidor.
- No escala entre múltiples workers/procesos.

Solución: migrar a modelos Django con base de datos real.

5.3. Riesgos de concurrencia

Variables globales (`secuencia_eventos`, `db_eventos`) pueden dar condiciones de carrera en ambientes concurrentes.

Solución:

- Usar transacciones en base de datos.
- Delegar ids autoincrementales al motor SQL.

5.4. Validaciones incompletas

Actualmente se valida fecha no vacía y capacidad mínima. Faltan casos:

- Formato de fecha.
- Fecha en pasado.
- Nombre vacío o demasiado corto.
- Evento sin servicios.

5.5. Tipado y compatibilidad de versiones

Uso de `tuple[bool, str]` puede fallar en versiones antiguas de Python (<3.9). **Solución:** usar `Tuple[bool, str]` desde `typing` para compatibilidad amplia.

5.6. Mantenibilidad del front

`dashboard.html` mezcla componentes visuales y etiquetas de patrón (didácticas). Riesgo: dificultad para internacionalización y cambios de branding.

Solución:

- Separar textos en archivo de traducción.
- Componentizar plantillas con `{% include% }`.

6. Flujo de ejecución resumido

6.1. Crear conferencia

1. Usuario hace clic en “Conferencia”.
2. Se invoca `crear_evento_builder(..., tipo='conferencia')`.
3. Director usa `EventoConferenciaBuilder`.

4. Builder agrega coffee break + streaming adaptado.
5. Cadena valida fecha y capacidad.
6. Se guarda en memoria y se redirige al panel.

6.2. Exportar JSON

1. Usuario abre `?export=json`.
2. Vista principal crea `ExportadorAPIJSON`.
3. Bridge transforma eventos a payload JSON.
4. Se responde `JsonResponse`.

7. Conclusión

El sistema demuestra correctamente la aplicación práctica de múltiples patrones de diseño en un caso de gestión de eventos. La arquitectura está orientada a extensibilidad, con separación de responsabilidades y uso de vistas claras.

Los principales problemas observables no invalidan el diseño, pero sí marcan el camino de evolución hacia un entorno productivo.